

MSBuild Command-Line Reference

Visual Studio 2015

When you use MSBuild.exe to build a project or solution file, you can include several switches to specify various aspects of the process.

Syntax

```
MSBuild.exe [Switches] [ProjectFile]
```

Arguments

Argument	Description
<i>ProjectFile</i>	Builds the targets in the project file that you specify. If you don't specify a project file, MSBuild searches the current working directory for a file name extension that ends in "proj" and uses that file. You can also specify a Visual Studio solution file for this argument.

Switches

Switch	Short form	Description
/help	/? or /h	Display usage information. The following command is an example: msbuild.exe /?
/detailedsummary	/ds	Show detailed information at the end of the build log about the configurations that were built and how they were scheduled to nodes.
/ignoreprojectextensions: <i>extensions</i>	/ignore: <i>extensions</i>	Ignore the specified extensions when determining which project file to build. Use a semicolon or a comma to separate multiple extensions, as the following example shows: /ignoreprojectextensions:.vcproj;.sln
/maxcpucount[: <i>number</i>]	/m[: <i>number</i>]	Specifies the maximum number of concurrent processes to use when building. If you don't include this switch, the default value is 1. If you include

		this switch without specifying a value, MSBuild will use up to the number of processors in the computer. For more information, see Building Multiple Projects in Parallel with MSBuild. The following example instructs MSBuild to build using three MSBuild processes, which allows three projects to build at the same time: <pre>msbuild myproject.proj /maxcpucount:3</pre>
/noautoresponse	/noautorsp	Don't include any MSBuild.rsp files automatically.
/nodeReuse:value	/nr:value	Enable or disable the re-use of MSBuild nodes. You can specify the following values: • True. Nodes remain after the build finishes so that subsequent builds can use them (default). • False. Nodes don't remain after the build completes. A node corresponds to a project that's executing. If you include the /maxcpucount switch, multiple nodes can execute concurrently.
/nologo		Don't display the startup banner or the copyright message.
/preprocess[:filepath]	/pp[:filepath]	Create a single, aggregated project file by inlining all the files that would be imported during a build, with their boundaries marked. You can use this switch to more easily determine which files are being imported, from where the files are being imported, and which files contribute to the build. When you use this switch, the project isn't built. If you specify a <i>filepath</i>, the aggregated project file is output to the file. Otherwise, the output appears in the console window. For information about how to use the Import element to insert a project file into another project file, see Import Element (MSBuild) and How to: Use the Same Target in Multiple Project Files.
/property:name=value	/p:name=value	Set or override the specified project-level properties, where <i>name</i> is the property name and <i>value</i> is the property value. Specify each property separately, or use a semicolon or comma to separate multiple properties, as the following example shows: <pre>/property:WarningLevel=2;OutDir=bin\Debug</pre>
/target:targets	/t:targets	Build the specified targets in the project. Specify each target separately, or use a semicolon or comma to separate multiple targets, as the following example shows: <pre>/target:Resources;Compile</pre> If you specify any targets by using this switch, they are run instead of any targets in the DefaultTargets attribute in the project file. For more information, see Target Build Order and How to: Specify Which Target to Build First.

		A target is a group of tasks. For more information, see MSBuild Targets .
<code>/toolsversion: version</code>	<code>/tv:version</code>	Specifies the version of the Toolset to use to build the project, as the following example shows: <code>/toolsversion:3.5</code> By using this switch, you can build a project and specify a version that differs from the version that's specified in the Project Element (MSBuild). For more information, see Overriding ToolsVersion Settings. For MSBuild 4.5, you can specify the following values for <i>version</i>: 2.0, 3.5, and 4.0. If you specify 4.0, the VisualStudioVersion build property specifies which sub-toolset to use. For more information, see the Sub-toolsets section of MSBuild Toolset (ToolsVersion). A Toolset consists of tasks, targets, and tools that are used to build an application. The tools include compilers such as csc.exe and vbc.exe. For more information about Toolsets, see MSBuild Toolset (ToolsVersion), Standard and Custom Toolset Configurations, and MSBuild Multitargeting Overview.  Note The toolset version isn't the same as the target framework, which is the version of the .NET Framework on which a project is built to run. For more information, see MSBuild Target Framework and Target Platform.
<code>/validate: [schema]</code>	<code>/val[schema]</code>	Validate the project file and, if validation succeeds, build the project. If you don't specify <i>schema</i>, the project is validated against the default schema. If you specify <i>schema</i>, the project is validated against the schema that you specify. The following setting is an example: <code>/validate:MyExtendedBuildSchema.xsd</code>
<code>/verbosity:lev el</code>	<code>/v:level</code>	Specifies the amount of information to display in the build log. Each logger displays events based on the verbosity level that you set for that logger. You can specify the following verbosity levels: q[uiet], m[inimal], n[ormal], d[etailed], and diag[nostic]. The following setting is an example: <code>/verbosity:quiet</code>
<code>/version</code>	<code>/ver</code>	Display version information only. The project isn't built.
<code>@file</code>		Insert command-line switches from a text file. If you have multiple files, you specify them separately. For more information, see MSBuild Response Files .

Switches for Loggers

Switch	Short form	Description
/consoleloggerparameters: <i>parameters</i>	/clp: <i>parameters</i>	Pass the parameters that you specify to the console logger, which displays build information in the console window. You can specify the following parameters: • PerformanceSummary. Show the time that's spent in tasks, targets, and projects. • Summary. Show the error and warning summary at the end. • NoSummary. Don't show the error and warning summary at the end. • ErrorsOnly. Show only errors. • WarningsOnly. Show only warnings. • NoItemAndPropertyList. Don't show the list of items and properties that would appear at the start of each project build if the verbosity level is set to diagnostic. • ShowCommandLine. Show TaskCommandLineEvent messages. • ShowTimestamp. Show the timestamp as a prefix to any message. • ShowEventId. Show the event ID for each started event, finished event, and message. • ForceNoAlign. Don't align the text to the size of the console buffer. • DisableConsoleColor. Use the default console colors for all logging messages. • DisableMPLogging. Disable the multiprocessor logging style of output when running in non-multiprocessor mode. • EnableMPLogging. Enable the multiprocessor logging style even when running in non-multiprocessor mode. This logging style is on by default. • Verbosity. Override the /verbosity setting for this logger. Use a semicolon or comma to separate multiple parameters, as the following example shows: <pre>/consoleloggerparameters:PerformanceSummary;NoSummary /verbosity:minimal</pre>
/distributedFileLogger	/dfl	Log the build output of each MSBuild node to its own file. The initial location for these files is the current directory. By default, the files are named "MSBuildNodeId.log". You can use the /fileLoggerParameters switch to specify the location of the files and other parameters for the fileLogger. If you name a log file by using the /fileLoggerParameters switch, the distributed logger will use that name as a template and append the node ID to that name when creating a log file for each node.
/distributedlogger: <i>central</i>	/dl: <i>central logger*forwarding logger</i>	Log events from MSBuild, attaching a different logger instance to each node. To specify multiple loggers, specify each logger separately. You use the logger syntax to specify a logger. For the logger syntax, see

<i>logger*</i> <i>forwarding logger</i>		the /logger switch below. The following examples show how to use this switch: <pre>/dl:XMLLogger,MyLogger,Version=1.0.2,Culture=neutral</pre> <pre>/dl:MyLogger,C:\My.dll*ForwardingLogger,C:\Logger.dll</pre>
/fileLogger <i>[number]</i>	/fl[<i>number</i>]	Log the build output to a single file in the current directory. If you don't specify <i>number</i>, the output file is named msbuild.log. If you specify <i>number</i>, the output file is named msbuildn.log, where n is <i>number</i>. <i>Number</i> can be a digit from 1 to 9. You can use the /fileLoggerParameters switch to specify the location of the file and other parameters for the fileLogger.
/fileloggerpar ameters: <i>[number]</i> <i>parameters</i>	/flp:[<i>number</i>] parameters	Specifies any extra parameters for the file logger and the distributed file logger. The presence of this switch implies that the corresponding /filelogger[<i>number</i>] switch is present. <i>Number</i> can be a digit from 1 to 9. You can use all parameters that are listed for /consoleloggerparameters. You can also use one or more of the following parameters: • LogFile. The path to the log file into which the build log is written. The distributed file logger prefixes this path to the names of its log files. • Append. Determines whether the build log is appended to the log file or overwrites it. When you set the switch, the build log is appended to the log file. When the switch is not present, the contents of an existing log file are overwritten. If you include the append switch, no matter whether it is set to true or false, the log is appended. If you do not include the append switch, the log is overwritten. In this case the file is overwritten: <code>msbuild myfile.proj /l:FileLogger,Microsoft.Build.Engine;logfile=MyLog.log</code> In this case the file is appended: <code>msbuild myfile.proj /l:FileLogger,Microsoft.Build.Engine;logfile=MyLog.log;append=true</code> In this case the file is appended: <code>msbuild myfile.proj /l:FileLogger,Microsoft.Build.Engine;logfile=MyLog.log;append=false</code> • Encoding. Specifies the encoding for the file (for example, UTF-8, Unicode, or ASCII). The following example generates separate log files for warnings and errors: <pre>/flp1:logfile=errors.txt;errorsonly</pre> <pre>/flp2:logfile=warnings.txt;warningsonly</pre> The following examples show other possibilities:

		<pre> /fileLoggerParameters:LogFile=MyLog.log;Append; Verbosity=diagnostic;Encoding=UTF-8 /flp:Summary;Verbosity=minimal;LogFile=msbuild.sum /flp1:warningsonly;logfile=msbuild.wrn /flp2:errorsonly;logfile=msbuild.err </pre>
<pre> /logger: logger </pre>	<pre> /!:logger </pre>	Specifies the logger to use to log events from MSBuild. To specify multiple loggers, specify each logger separately. Use The following syntax for <i>logger</i>: <pre>[LoggerClass,]LoggerAssembly[;LoggerParameters]</pre> Use the following syntax for <i>LoggerClass</i>: <pre>[PartialOrFullNamespace.]LoggerClassName</pre> You don't have to specify the logger class if the assembly contains exactly one logger. Use the following syntax for <i>LoggerAssembly</i>: <pre>{AssemblyName[,StrongName] AssemblyFile}</pre> Logger parameters are optional and are passed to the logger exactly as you enter them. The following examples use the /logger switch. <pre> /logger:XMLLogger,MyLogger,Version=1.0.2,Culture=neutral /logger:XMLLogger,C:\Loggers\MyLogger.dll;OutputAsHTML </pre>
<pre> /noconsolelo gger </pre>	<pre> /noconlog </pre>	Disable the default console logger, and don't log events to the console.

Example

The following example builds the **rebuild** target of the **MyProject.proj** project.

```
MSBuild.exe MyProject.proj /t:rebuild
```

Example

You can use MSBuild.exe to perform more complex builds. For example, you can use it to build specific targets of specific projects in a solution. The following example rebuilds the project **NotInSolutionFolder** and cleans the project **InSolutionFolder**, which is in the **NewFolder** solution folder.

```
msbuild SlnFolders.sln /t:NotInSolutionFolder:Rebuild;NewFolder\InSolutionFolder:Clean
```

See Also

MSBuild Reference

Common MSBuild Project Properties

;

© 2016 Microsoft

